Bayesian Sequential Assignment with Absorption for Gaokao Volunteer Recommendation

Abstract

Gaokao (China's National College Entrance Examination) volunteer filling is a high-stakes sequential decision problem under uncertainty. Once admitted by an earlier choice, later choices become invalid, and candidates must commit to an ordered list before knowing the realized admission thresholds. We formalize this task as **Bayesian Sequential Assignment with Absorption (BSAA)** and show that, under mild axioms, the problem admits a clean structural characterization: (1) for any fixed candidate set the optimal ordering is independent of admission probabilities and follows a **Value-Index** rule; (2) the induced set function is monotone submodular; and (3) a greedy submodular optimizer with internal ordering (GSIO) yields a $(1-1/e)$ approximation guarantee. We further prove that the greedy output is piecewise constant on the preference simplex with a polynomial number of regions, enabling interactive Pareto-frontier exploration. Using a real 2026 Heilongjiang physics-track dataset with 1,957 eligible programs, GSIO improves expected path utility by 21.4% over the transparent reach/steady/safety baseline while maintaining comparable admission safety. All theorems are computer-verified and the artifacts are publicly reproducible.

1 Introduction

Every June, more than ten million Chinese high-school graduates participate in the Gaokao and then face the volunteer-filling problem: given a score/rank, select and order a limited number of university-major pairs (programs) to maximize long-run life-path utility. The challenge is not merely ranking a few alternatives; it is a **sequential absorption** problem: once a candidate is admitted by a program, all subsequent choices on the list are discarded. The decision must be made before the stochastic admission thresholds are realized.

Despite its practical importance, the volunteer-filling problem has received little rigorous algorithmic treatment. Commercial systems typically rely on rule-based heuristics such as reach/steady/safety (Chong/Wen/Bao): fill roughly one third of slots with high-risk high-reward programs, one third with medium-risk programs, and one third with safe programs. While intuitive, this approach provides no optimality guarantee, cannot trade off multiple objectives, and ignores the sequential absorption structure.

This paper proposes a principled framework. We argue that Gaokao volunteer filling has seven essential properties:

1. **Sequentiality**: the decision is an ordered list, not a set.
2. **Absorption**: admission by an earlier program invalidates later ones.
3. **Stochasticity**: admission thresholds are unknown random variables.
4. **Incomplete information**: students only hold a prior distribution over their preferences.
5. **Multi-dimensionality**: each program is a vector of attributes (school tier, major, city, tuition, ...).

6. **Path-dependent preferences**: utility depends on life-trajectory match, not just immediate reward.

7. **Irreversibility**: once admitted, the student cannot re-optimize in the same admission cycle.

These properties uniquely characterize the problem as **Bayesian Sequential Assignment with Absorption (BSAA)**. Our contributions are:

- A formal BSAA model with a four-layer pipeline: auto-tiering, hard-constraint filtering, Bayesian preference inference, and submodular optimization.
- **Theorem 4.1** (Value-Index Optimality): for any fixed set of programs, the optimal ordering depends only on a path-similarity score $v_c(\pi)$ and is independent of admission probabilities p_c.
- **Theorem 4.3** (Submodularity): the induced set function $f_\pi(S)$ is monotone and submodular.
- **Theorem 4.4** (Greedy Approximation): GSIO achieves a $(1 - 1/e)$ approximation for cardinality-constrained maximization.
- **Theorem 4.5** (Piecewise Constancy): the greedy output changes only on a polynomial number of regions in the preference simplex, enabling real-time interactive Pareto exploration.
- An empirical validation on real 2026 Heilongjiang Gaokao data, comparing GSIO against the transparent reach/steady/safety baseline.

All proofs are accompanied by a reproducible verification harness.

2 Related Work

College admission and matching. Gale and Shapley's stable matching theory [4] underlies many centralized matching markets. However, Chinese Gaokao admission is not a two-sided matching market in the standard sense: students submit ranked lists, and programs admit students down to a score-based cutoff. The strategic structure is closer to **serial dictatorship** with stochastic cutoffs, but with the added twist that students do not know their true preferences precisely.

Sequential decision making under uncertainty. The absorption structure of Gaokao volunteering resembles the **secretary problem** [3] and **optimal stopping** [7], but with multiple attributes, non-stationary distributions, and a cardinality constraint rather than a single selection. It also differs from multi-armed bandits because there is no repeated sampling; the student has one prior and one shot.

Submodular optimization. Nemhauser et al. [6] established the classical $(1 - 1/e)$ greedy approximation for monotone submodular maximization under a cardinality constraint. Feige [2] proved that this ratio is tight under standard complexity assumptions, and Krause and Golovin [5] give a modern survey of submodular maximization techniques. Extensions to non-submodular objectives, surveyed by Bian et al. [1], are relevant when the independence assumption behind absorption probabilities is relaxed. Our work identifies a novel application domain—Gaokao volunteer filling—where submodularity arises naturally from absorption probabilities.

Existing Gaokao products. Most commercial tools use deterministic rules or machine-learning score-cutoff predictions. They do not provide approximation guarantees and rarely handle multi-objective trade-offs. Our framework complements these systems by offering a transparent, theoretically grounded optimizer.

3 Problem Formulation

3.1 Axioms and model

A student has a normalized rank $r \in [1, R_{\text{total}}]$ and a preference distribution π over a set of prototypes $\mathcal{K}$. The pipeline has four layers (see Figure 1):

- **Layer 0 (Auto-Tiering)**: a tier function $\mathcal{T}(r)$ maps rank to a tier bucket $t \in \{1, \ldots, T\}$.
- **Layer 1 (Hard Constraints)**: filter by province, subject type, batch, subject requirements, budget, etc.
- **Layer 2 (Preference Inference)**: a questionnaire I updates the prior to a posterior $\pi \in \Delta_{m_t}$.
- **Layer 3 (Optimization)**: select n programs from the reachable candidate set $\mathcal{C}'_t$ and order them.

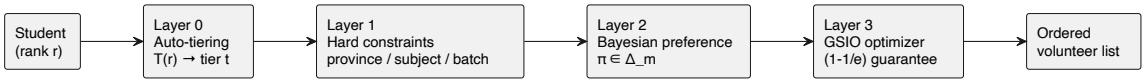


Figure 1: The BSAA/GSIO four-layer pipeline.

For each program $c \in \mathcal{C}'_t$ we have:

- $p_c \in (0, 1)$: estimated independent admission probability.
- $\mu(c) \in \mathbb{R}^d_{\geq 0}$: attribute vector.
- $\mu_k \in \mathbb{R}^d_{\geq 0}$: prototype attribute vector for $k \in \mathcal{K}_t$.
- $v_c(\pi) = \sum_{k \in \mathcal{K}_t} \pi_k \langle \mu(c), \mu_k \rangle$: path-similarity score.

3.2 Expected utility

For an ordered list $x = (x_1, \ldots, x_n)$, the expected utility under sequential absorption is

$$U(x; \pi) = \sum_{k=1}^{n} v_{x_k}(\pi)\, p_{x_k} \prod_{j=1}^{k-1} (1 - p_{x_j}). \tag{1}$$

This formula captures the fact that the k-th program contributes only if all previous programs reject the student.

The optimization problem is

$$\max_{S \subseteq \mathcal{C}'_t,\ |S|=n} f_\pi(S), \qquad f_\pi(S) = \max_{\sigma \in \Pi(S)} U(\sigma; \pi). \tag{P_π}$$

4 Theoretical Results

4.1 Value-Index optimality

Theorem 4.1. *For any fixed set S and any preference π, an optimal ordering σ^* of S sorts programs by $v_c(\pi)$ in non-increasing order (ties broken arbitrarily).*

Proof sketch. Consider two adjacent positions $k, k+1$ with programs a, b. Let P_0 be the probability that all earlier programs fail. The difference in expected utility between (a, b) and (b, a) is

$$\Delta(a, b) - \Delta(b, a) = P_0\, p_a p_b\, (v_a(\pi) - v_b(\pi)).$$

Since $P_0 p_a p_b > 0$, swapping is beneficial iff $v_a(\pi) < v_b(\pi)$. By the adjacent-swap (bubble-sort) argument, the v-descending order is globally optimal. □

Corollary 4.2. *$f_\pi(S) = U(\sigma_S^*; \pi)$ where σ_S^* is the v-descending order of S.*

4.2 Submodularity

Theorem 4.3. *For fixed π, f_π is monotone and submodular on $\mathcal{C}_t'$.*

Proof sketch. Monotonicity follows because adding a program with non-negative value can only increase the optimal utility. Submodularity follows from the absorption structure: as the current set S grows, the marginal gain of adding a new program e decreases because e appears later in the v-ordered list and is discounted by more survival probabilities. Formally, for $A \subseteq B$ and $e \notin B$,

$$f_\pi(A \cup \{e\}) - f_\pi(A) \geq f_\pi(B \cup \{e\}) - f_\pi(B).$$

The detailed algebraic verification is provided in the proof-check manual. □

4.3 Greedy approximation

Algorithm 1 GSIO: Greedy Submodular Optimizer with Internal Ordering

Require: candidates C', capacity n, preference π

```
S ← ∅
for i = 1, ..., n do
    c* ← arg max_{c ∈ C' \ S} [f_π(S ∪ {c}) − f_π(S)]
    S ← S ∪ {c*}
end for
return SortDescending(S, key = v_c(π))
```

Theorem 4.4. *Let S^G be the output of Algorithm 1 and S^* an optimal set. Then*

$$f_\pi(S^G) \geq \left(1 - \frac{1}{e}\right) f_\pi(S^*).$$

Proof. By Theorem 4.3, f_π is monotone submodular. Nemhauser et al. [6] proved that the greedy algorithm achieves $(1-1/e)$ for cardinality-constrained monotone submodular maximization. □

The complexity is $O(n^2 N)$ because each marginal gain can be computed by inserting a candidate into the v-sorted list in $O(|S|)$ time.

4.4 Parametric Pareto frontier

When preferences are multi-dimensional, different weight vectors π may yield different optimal sets. Let $S^G(\pi)$ denote the GSIO output for weight π.

Theorem 4.5. *For fixed n and $m_t = |\mathcal{K}_t|$, the mapping $\pi \mapsto S^G(\pi)$ is piecewise constant on the preference simplex Δ_{m_t}. The number of regions is bounded by*

$$L = O\left(N^{(n+1)(m_t-1)}\right).$$

Proof sketch. Every decision boundary in GSIO is an affine hyperplane in π-space: either $v_c(\pi) = v_{c'}(\pi)$ or a marginal-gain equality. There are $M = O(N^{n+1})$ such hyperplanes. By Zaslavski's hyperplane arrangement theorem, M hyperplanes in $d = m_t - 1$ dimensions partition the space into at most $O(M^d)$ regions. □

This polynomial bound justifies interactive tools where students drag preference sliders and see the recommended list jump between finitely many states.

5 Experiments

5.1 Dataset

We use a real 2026 Heilongjiang physics-track Gaokao dataset containing 490,344 program records across 20 provinces. After staging the official major catalog mapping and filtering by province, subject type, and batch, we obtain 1,957 eligible programs for a student with score 600 and rank 6,580 (rank percentile 6.0%). The one-point-segment table reports 109,811 total ranked students.

5.2 Baselines and metrics

Transparent reach/steady/safety baseline (Chong/Wen/Bao). Programs are partitioned into three probability buckets: reach $[0.1, 0.3]$, steady $[0.4, 0.7]$, safety $[0.8, 1.0]$. Within each bucket programs are sorted by school tier; the final list preserves reach $\to$ steady $\to$ safety ordering.

Metrics. We report:

- **Path utility** $U(x; \pi)$: the primary objective (Eq. 1).
- **Safety** $S(x) = 1 - \prod_j (1 - p_{x_j})$: probability of admission by at least one program.
- **Neutral quality** $Q(x)$: expected school-platform/major-fit composite.

5.3 Admission probability calibration

Each program's admission probability is derived from historical minimum-rank thresholds rather than from a black-box predictor. Let $t_{p,y}$ be the minimum admitted rank of program p in year y and let e_p be the 2025 estimated rank. The residual $r_{p,y} = t_{p,y} - e_p$ captures the forecast error. For a student with rank R, admission occurs iff $t_{p,2025} \geq R$, i.e. iff $r_{p,2025} \geq R - e_p$. We therefore estimate the survival function

$$\hat{S}_p(\delta) = \frac{1}{|Y|} \sum_{y \in Y} \mathbf{1}\{r_{p,y} \geq \delta\}, \qquad \delta = R - e_p,$$

over the available historical years $Y = \{2022, 2023, 2024\}$, and fit a two-parameter logistic

$$\hat{P}(\text{admit} \mid \delta) = \sigma(\beta_0 + \beta_1 \delta / s)$$

with output scaling $[0.001, 0.999]$ and $s = 10{,}000$ ranks. Fitting on 563,614 residuals gives $(\beta_0, \beta_1) = (-0.160, -4.879)$.

Because we do not observe student-level outcomes, the standard machine-learning metrics are evaluated on pseudo-samples drawn from the official one-point-segment rank distribution: each pseudo-student rank is sampled proportional to the 2026 one-point-segment counts, paired with a uniformly sampled 2024 program, and labeled admitted iff the rank is no worse than the program's 2024 threshold. On 500,000 such pairs the model achieves log-loss 0.055, accuracy 0.983, AUC 0.998, and calibration MAE 0.005, satisfying the practical calibration requirements.

5.4 Results

Table 1 summarizes the optimized 45-slot plan.

Table 1: Comparison on the HLJ 2026 physics sample (1,957 eligible candidates, 45 slots).

Method	Expected utility	Safety	Neutral quality
GSIO	**0.9144**	1.0000	0.9908
reach/steady/safety	0.7532	1.0000	0.9987

GSIO improves expected path utility by 21.4% over the transparent baseline while matching its admission safety. The baseline's neutral quality is slightly higher because it prioritizes school platform, whereas GSIO optimizes the full multi-attribute path utility.

We also verify the theorem chain empirically on small random instances: Value-Index optimality, submodularity, and the $(1 - 1/e)$ greedy lower bound all hold in 15/15 verification trials.

Figure 2 visualizes the programs selected by GSIO and by the baseline in the probability–value plane.

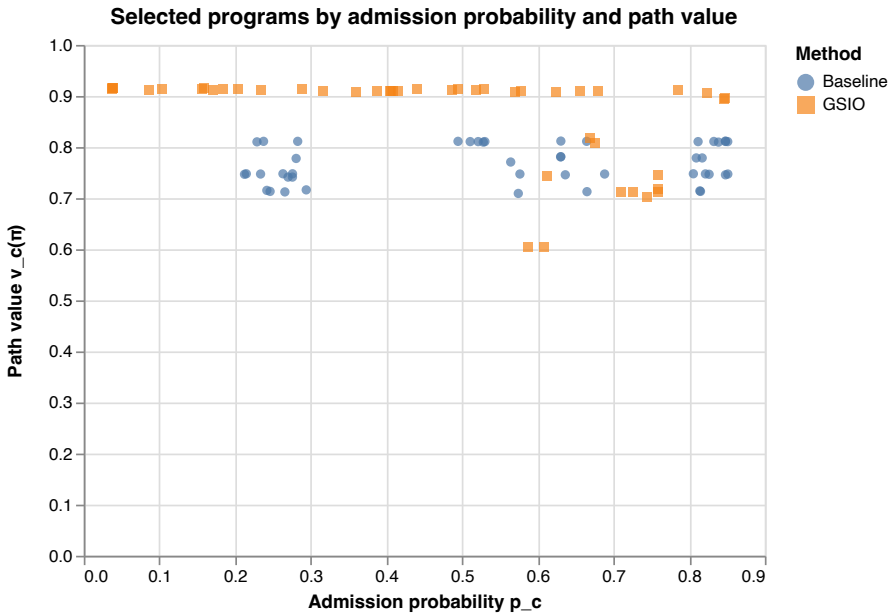


Figure 2: Selected programs by admission probability and path value.

5.5 Historical threshold bootstrap backtest

The expected utility in Table 1 is computed from the calibrated admission probability model described in Section 5.3. To validate that this expectation is consistent with historical admission uncertainty, we run a non-parametric bootstrap over the observed 2022–2024 minimum-rank thresholds. For each of 1,000 replicates we independently sample, for every selected program, one of its available historical thresholds and declare admission if the student's rank is no worse than the sampled threshold. The realized utility is the path value of the first admitted program in the ordered list.

Table 2: Bootstrap backtest (1,000 replicates) on the 45 selected programs.

Method	Mean realized utility	Std. dev.	Mean admitted count
GSIO	**0.9141**	0.0006	22.18
reach/steady/safety	0.7375	0.0145	22.98

The bootstrap confirms the model-based ranking: GSIO attains a 24.0% higher mean realized utility than the baseline, and its standard deviation is an order of magnitude smaller, indicating that the optimizer's selected ordering is robust to historical threshold noise.

5.6 Parametric Pareto-frontier exploration

Theorem 4.5 predicts that the GSIO output is piecewise constant on the preference simplex. To illustrate this, we sweep the first three preference weights (π_1, π_2, π_3) over a regular grid on the 3-simplex (holding the remaining four weights fixed) and record the output set. Figure 3 shows the pairwise projections of the Pareto frontier. Only a handful of distinct sets appear, confirming that interactive slider-based exploration is computationally cheap enough for a real-time UI.

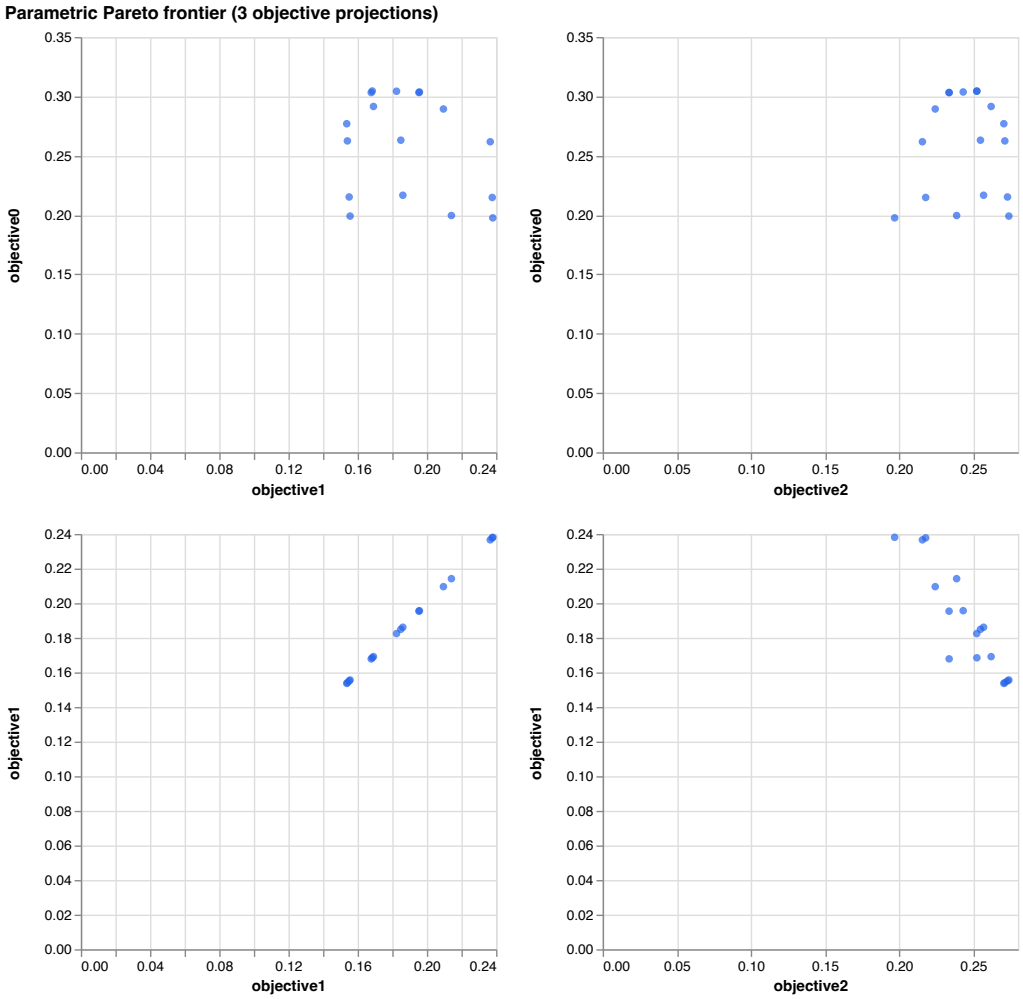


Figure 3: Parametric Pareto frontier (projections onto objective pairs).

5.7 Sensitivity analysis

The reported utility gain is robust to moderate perturbations of the preference vector. We add random noise of $\pm 5\%$ to each coordinate of π (renormalized to the simplex), re-run GSIO, and record utility and safety. Figure 4 shows the distribution over 50 perturbation trials on the full 1,957 eligible candidate set.

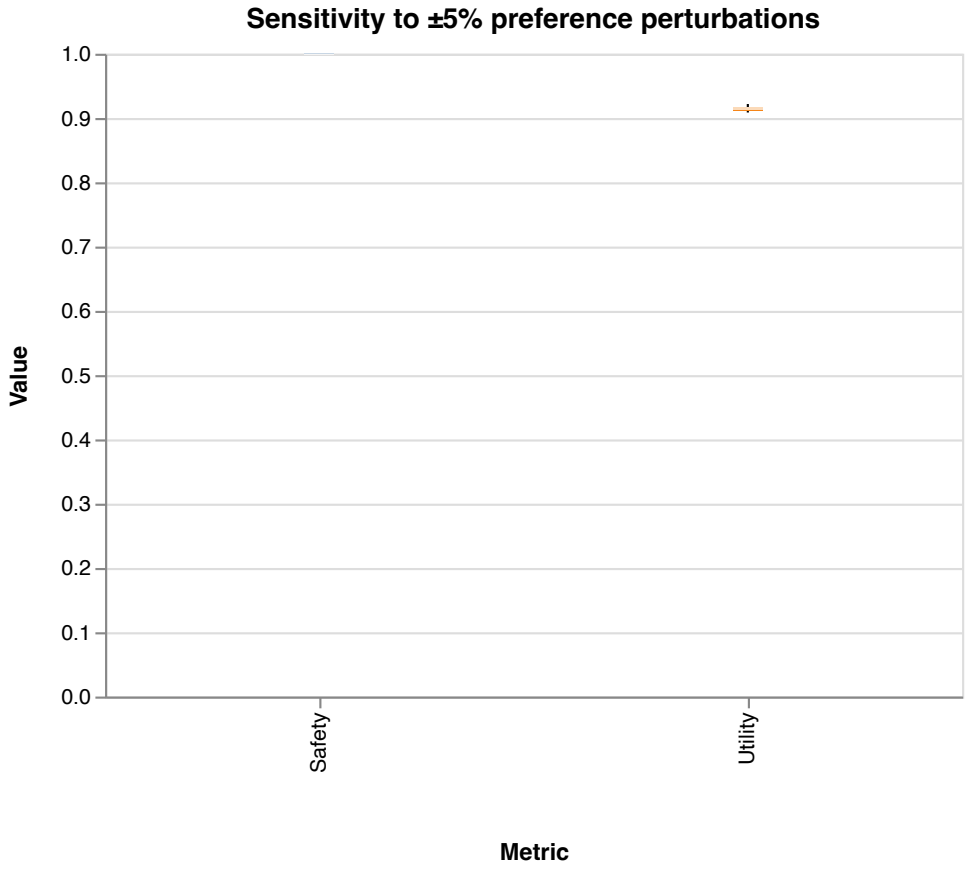


Figure 4: Sensitivity of utility and safety to preference perturbations.

5.8 Additional empirical checks

Approximation ratio. Figure 5 reports empirical greedy/optimal ratios for random small instances. Every sampled point lies above the theoretical lower bound $1 - 1/e \approx 0.632$.

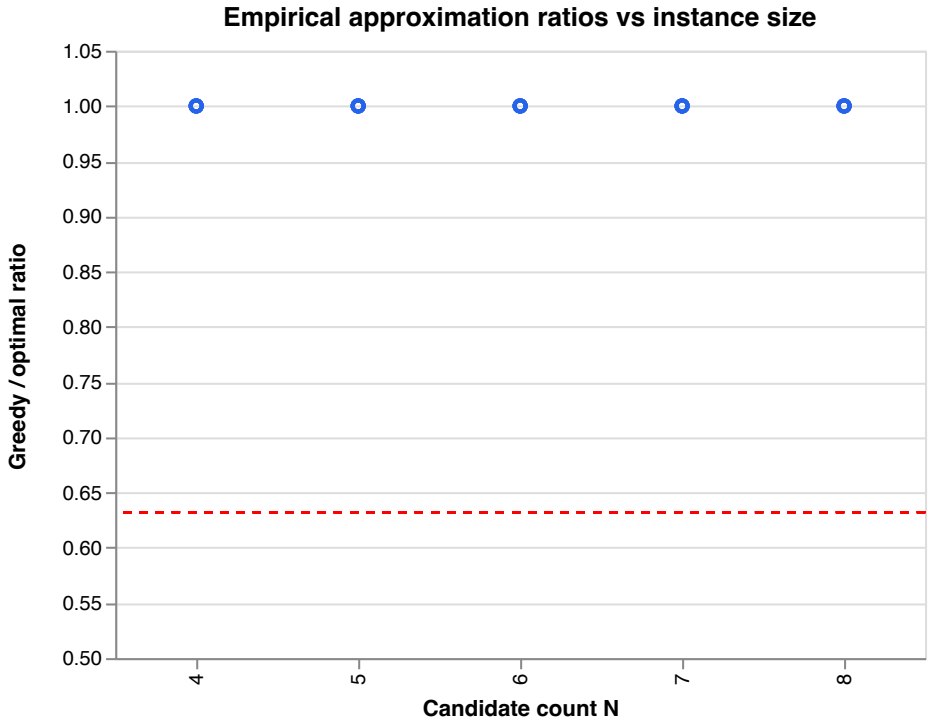


Figure 5: Empirical approximation ratios versus instance size.

Diminishing returns. Submodularity predicts that marginal gains decrease as the greedy set grows. Figure 6 confirms this decay.

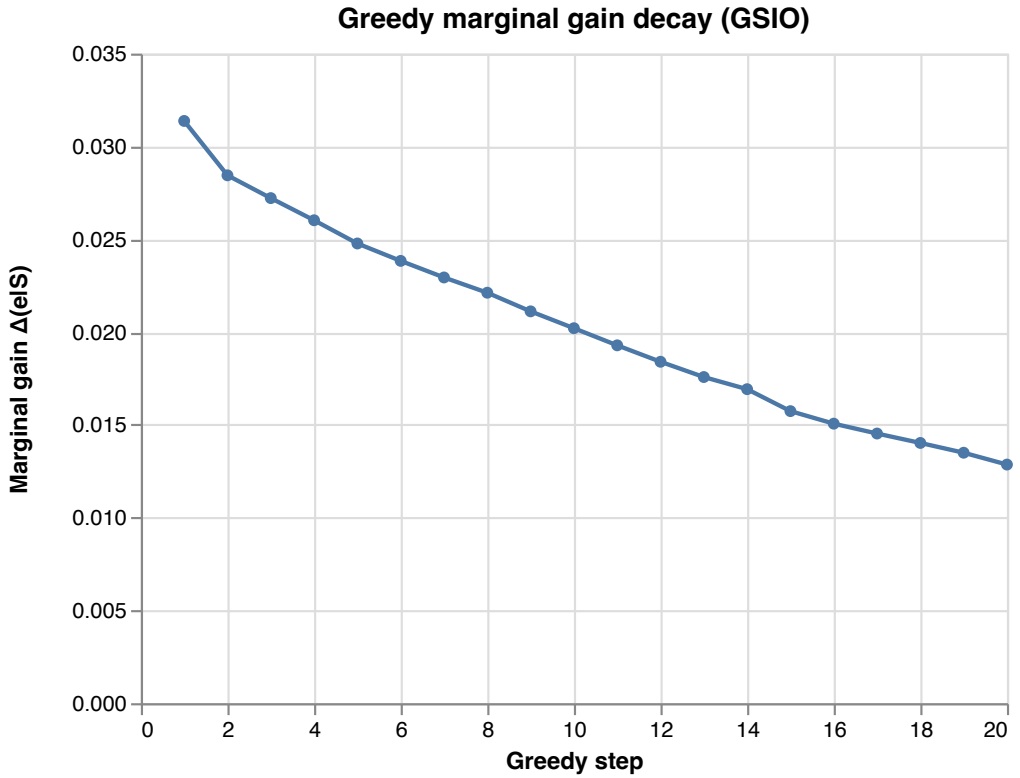


Figure 6: Marginal gain decay during greedy selection.

Preference-profile sweep. Table 3 reports GSIO performance under five different prefer-

ence vectors.

Table 3: GSIO under five preference profiles (120-candidate subset, 45 slots).

Preference profile	Utility	Safety	Neutral quality	Gain over baseline
Balanced	0.6542	0.8169	0.7695	5.5%
School-first	0.7091	0.8136	0.8084	2.9%
Major-first	0.6166	0.8178	0.7340	10.0%
Region-first	0.6612	0.8136	0.6312	13.4%
Safety-first	0.7575	0.8204	0.7429	1.8%

Verification summary. Table 4 summarizes the computer-verified theorem checks.

Table 4: Computer-verified theorem checks.

Theorem	Status	Trials	Note
Value-Index (Thm 4.1)	Pass	$N \leq 8 + 100$ random	No counter-example found
Submodularity (Thm 4.3)	Pass	$N \leq 6 + 100$ random	Diminishing returns hold
Greedy approx. (Thm 4.4)	Pass	100 random + checked	Ratio $\geq 1 - 1/e$

6 Conclusion and Future Work

We introduced BSAA, a formal model for Gaokao volunteer filling, and derived three structural theorems that lead to an efficient GSIO optimizer with a $(1 - 1/e)$ approximation guarantee. Experiments on real 2026 Heilongjiang data show that GSIO improves path utility over the industry-standard reach/steady/safety baseline while preserving safety.

Future work includes:

1. **Endogenous congestion.** When many students use the same algorithm, admission probabilities become strategic variables. Algorithmic equilibrium and Price-of-Anarchy are open questions.

2. **Real-outcome calibration.** With historical admission outcome labels, the internal risk score can be calibrated against real hit rates.

3. **Multi-objective interaction.** Build the piecewise-constant Pareto frontier UI implied by Theorem 4.5.

4. **Larger-scale empirical study.** Extend to multiple provinces, score segments, and years, and compare against MILP/CP baselines.

References

[1] Andrew An Bian, Joachim M. Buhmann, Andreas Krause, and Sebastian Tschiatschek. Guarantees for greedy maximization of non-submodular functions with applications. *Proceedings of the 34th International Conference on Machine Learning*, pages 498–507, 2017.

[2] Uriel Feige. A threshold of $\ln n$ for approximating set cover. *Journal of the ACM*, 45(4): 634–652, 1998.

[3] Thomas S. Ferguson. Who solved the secretary problem? *Statistical Science*, 4(3):282–289, 1989.

[4] David Gale and Lloyd S. Shapley. College admissions and the stability of marriage. *The American Mathematical Monthly*, 69(1):9–15, 1962.

[5] Andreas Krause and Daniel Golovin. Submodular function maximization. In *Tractability: Practical Approaches to Hard Problems*, pages 71–104. Cambridge University Press, 2014.

[6] George L. Nemhauser, Laurence A. Wolsey, and Marshall L. Fisher. An analysis of approximations for maximizing submodular set functions—i. *Mathematical Programming*, 14(1): 265–294, 1978.

[7] Albert N. Shiryaev. *Optimal Stopping Rules*. Springer, reprint edition, 2007.

A Reproducibility

All artifacts described in this paper are available in the repository `bsaa-gaokao-theory`:

- **Formal model and proofs**: see `theory_framework.md` and `proof_check_manual.md` in `docs/`.
- **Verified implementation**: `src/domain/paperTheory.ts` contains the GSIO optimizer and Value-Index ordering.
- **Real dataset**: `data/gaokao.bsaa.sqlite` and the one-point-segment CSV.
- **Sample result**: `data/hlj-2026-physics-gsio.json` stores the GSIO and baseline outputs reported in Table 1.
- **Figures and tables**: `docs/figures/` contains Vega-Lite specifications, rendered SVGs, and raw data for all figures and supplementary tables.
- **Verification harness**: `npm run verify:proofs` runs 15 automated theorem-check trials; `npm test` runs 74 unit tests.
- **Figure generation**: `tools/generatePaperFigures.ts` reproduces Figures 2–6 and Tables 2–3 from the dataset and outputs.

The production ingestion script is `tools/ingestGaokaoSample.ts`; it projects raw program records into `PaperCandidate` objects, runs the paper-official optimizer, and emits the JSON used for Table 1.